In [166…
```
#import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import sqlite3

import sqlite3

print("SQLite version:", sqlite3.sqlite_version)
```

```
SQLite version: 3.43.1
```

In [167…
```
#import database/data
import sqlite3
import pandas as pd

# Connect to database
conn = sqlite3.connect("C:/Users/ADMIN/Downloads/customer_churn.db")

# Get all table names
sql_query = """
SELECT name
FROM sqlite_master
WHERE type='table'
"""

tables = pd.read_sql(sql_query, conn)

print("Tables in database:")
print(tables)


# Create dataframe for each table
for table_name in tables['name']:
    df = pd.read_sql(f'SELECT * FROM "{table_name}"', conn)

    globals()[f"df_{table_name}"] = df

    print(f"Created dataframe: df_{table_name}")
    print(df.head())
    print()

conn.close()
```

```
Tables in database:
             name
0      db_customer
1  db_subscription
2       db_support
Created dataframe: df_db_customer
   customerid    name country        state  gender                  dob  \
0  0002-ORFBO  keshav   India  Maharashtra    Male  1982-04-12 00:00:00
1  0003-MKNFE  raghav   India    Karnataka    Male  1995-11-23 00:00:00
2  0004-TLHLJ  lalita   India        Delhi  Female  1978-02-15 00:00:00
3  0011-IGKFF   mohan   India     Nagaland    Male  2001-08-30 00:00:00
4  0013-EXCHZ    mira   India        Delhi  Female  1990-05-05 00:00:00

  interests pincode
0    travel    None
1       NaN    None
2     movie    None
3       NaN    None
4     drama    None

Created dataframe: df_db_subscription
   customerid subscription_start_date subscription_type renewal_date  \
0  0002-ORFBO              2021-03-15          Refferal   2025-03-15
1  0003-MKNFE              2020-08-01              Paid   2024-08-01
2  0004-TLHLJ              2022-11-20           Organic   2025-11-20
3  0011-IGKFF              2019-05-10              Paid   2025-05-10
4  0013-EXCHZ              2023-01-05          Refferal   2024-01-05

  plan_type contract_type cancellation_date     cancellation_reason  \
0  Standard        Annual               NaN                     NaN
1   Premium        Annual        2024-09-10  Switched to competitor
2     Basic       Monthly               NaN                     NaN
3   Premium        Annual               NaN                     NaN
4  Standard       Monthly        2024-02-28           Too expensive

   monthly_charges  cltv  churn_score
0            13.99   627           12
1            12.99  1150           91
2             6.99   210           34
3            22.99  1725            8
4            13.99   195           88

Created dataframe: df_db_support
   customerid       complaint_date escalations  csat_score col_1  \
0  0003-MKNFE  2024-08-28 00:00:00           N          60  None
1  0003-MKNFE  2024-08-28 00:00:00           Y          10  None
2  0013-EXCHZ  2024-01-20 00:00:00           Y          20  None
3  0013-MHZWF  2025-03-18 00:00:00           N          90  None
4  0013-SMEOE  2024-11-01 00:00:00           N          30  None

            comment
0     service issue
1    demaned refund
2               NaN
3  guidance to renew
```

```
4                  NaN
```

In [168…
```
# print table names and column names
conn = sqlite3.connect("C:/Users/ADMIN/Downloads/customer_churn.db")

for table_name in tables['name']:
                        print(f"\nTable Name: {table_name}")
                        #Get column information
                        columns_query = f"PRAGMA table_info({table_name});"
                        columns = pd.read_sql(columns_query, conn)
                        print("columns:")
                        print(columns['name'].tolist())
# close connection
conn.close()
```

```
Table Name: db_customer
columns:
['customerid', 'name', 'country', 'state', 'gender', 'dob', 'interests', 'pincode']

Table Name: db_subscription
columns:
['customerid', 'subscription_start_date', 'subscription_type', 'renewal_date', 'plan
_type', 'contract_type', 'cancellation_date', 'cancellation_reason', 'monthly_charge
s', 'cltv', 'churn_score']

Table Name: db_support
columns:
['customerid', 'complaint_date', 'escalations', 'csat_score', 'col_1', 'comment']
```

In [169…
```
#2.data cleaning
df_db_customer.head()
df_db_customer.tail()
```

Out[169…

	customerid	name	country	state	gender	dob	interests	pincode
16	0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19 00:00:00	NaN	None
17	0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02 00:00:00	NaN	None
18	0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30 00:00:00	NaN	None
19	0023-HGHWL	rishabh	India	Uttar Pradesh	Men	1991-05-14 00:00:00	NaN	None
20	0023-UYUPN	sudevi	India	Maharashtra	Women	1977-10-06 00:00:00	NaN	None

In [170…
```
df_db_customer.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 8 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   customerid  21 non-null     str
 1   name        21 non-null     str
 2   country     18 non-null     str
 3   state       21 non-null     str
 4   gender      21 non-null     str
 5   dob         21 non-null     str
 6   interests   4 non-null      str
 7   pincode     0 non-null      object
dtypes: object(1), str(7)
memory usage: 1.4+ KB
```

In [171…
```
#a.rename col - name
#b.drop columns - interest and pincode
#c. change data type - dob
#d. data standardization - gender
#e. fix missing values - country
```

In [172…
```
#a. rename col - name
df_db_customer.rename(columns = {'name':'customer_name'}, inplace=True)
```

In [173…
```
#b. drop columns - interest and pincode
# df_db_customer.drop(df_db_customer.columns[-2:],axis=1)
# df_db_customer.columns[-2:]

df_db_customer.drop(columns=['interests','pincode'], inplace=True)
```

In [174…
```
df_db_customer.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   customerid     21 non-null     str
 1   customer_name  21 non-null     str
 2   country        18 non-null     str
 3   state          21 non-null     str
 4   gender         21 non-null     str
 5   dob            21 non-null     str
dtypes: str(6)
memory usage: 1.1 KB
```

In [175…
```
 #change the data type dob
df_db_customer['dob'] = pd.to_datetime(df_db_customer['dob'])
```

In [176…
```
#d. data standardization - gender
df_db_customer['gender'] = df_db_customer['gender'].replace({'Men':'Male','Women':'
```

In [177…
```
df_db_customer['gender'].unique()
```

Out[177…
```
<StringArray>
['Male', 'Female']
Length: 2, dtype: str
```

In [178…
```
#e. fix missing values - country
df_db_customer[df_db_customer['country'].isna()]
```

Out[178…

	customerid	customer_name	country	state	gender	dob
5	0013-MHZWF	durga	NaN	Delhi	Female	1988-12-10
8	0015-UOCOJ	maya	NaN	Kathmandu	Female	1985-07-07
12	0018-NYROU	chitra	NaN	Telangana	Female	2004-12-01

In [179…
```
#country and state - unique value pair
state_country_mapping = df_db_customer.dropna(subset=['country']).set_index('state'
df_db_customer['country'] = df_db_customer['country'].fillna(df_db_customer['state'
```

In [180…
```
df_db_customer[df_db_customer['country'].isna()]
```

Out[180…

customerid	customer_name	country	state	gender	dob

In [181…
```
df_db_subscription.head()
```

Out[181…

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	contract
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	A
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	A
2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	M
3	0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	A
4	0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	M

In [182…
```
df_db_subscription.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   customerid               21 non-null     str
 1   subscription_start_date  21 non-null     str
 2   subscription_type        21 non-null     str
 3   renewal_date             21 non-null     str
 4   plan_type                21 non-null     str
 5   contract_type            21 non-null     str
 6   cancellation_date        6 non-null      str
 7   cancellation_reason      6 non-null      str
 8   monthly_charges          21 non-null     float64
 9   cltv                     21 non-null     int64
 10  churn_score              21 non-null     int64
dtypes: float64(1), int64(2), str(8)
memory usage: 1.9 KB
```

In [183…

```
date_col = ['subscription_start_date', 'renewal_date', 'cancellation_date']

df_db_subscription[date_col] = df_db_subscription[date_col].apply(
    pd.to_datetime, errors='coerce'
)
df_db_subscription.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   customerid               21 non-null     str
 1   subscription_start_date  21 non-null     datetime64[us]
 2   subscription_type        21 non-null     str
 3   renewal_date             21 non-null     datetime64[us]
 4   plan_type                21 non-null     str
 5   contract_type            21 non-null     str
 6   cancellation_date        6 non-null      datetime64[us]
 7   cancellation_reason      6 non-null      str
 8   monthly_charges          21 non-null     float64
 9   cltv                     21 non-null     int64
 10  churn_score              21 non-null     int64
dtypes: datetime64[us](3), float64(1), int64(2), str(5)
memory usage: 1.9 KB
```

In [184…

```
print(df_db_subscription.columns.tolist())
```

```
['customerid', 'subscription_start_date', 'subscription_type', 'renewal_date', 'plan
_type', 'contract_type', 'cancellation_date', 'cancellation_reason', 'monthly_charge
s', 'cltv', 'churn_score']
```

In [185…

```
df_db_support.head()
```

Out[185…

	customerid	complaint_date	escalations	csat_score	col_1	comment
0	0003-MKNFE	2024-08-28 00:00:00	N	60	None	service issue
1	0003-MKNFE	2024-08-28 00:00:00	Y	10	None	demaned refund
2	0013-EXCHZ	2024-01-20 00:00:00	Y	20	None	NaN
3	0013-MHZWF	2025-03-18 00:00:00	N	90	None	guidance to renew
4	0013-SMEOE	2024-11-01 00:00:00	N	30	None	NaN

In [186…
```
df_db_support.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   customerid      9 non-null      str
 1   complaint_date  9 non-null      str
 2   escalations     9 non-null      str
 3   csat_score      9 non-null      int64
 4   col_1           0 non-null      object
 5   comment         4 non-null      str
dtypes: int64(1), object(1), str(4)
memory usage: 564.0+ bytes
```

In [187…
```
df_db_support.drop(columns=['col_1','comment'], inplace=True)
```

In [188…
```
df_db_support.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 4 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   customerid      9 non-null      str
 1   complaint_date  9 non-null      str
 2   escalations     9 non-null      str
 3   csat_score      9 non-null      int64
dtypes: int64(1), str(3)
memory usage: 420.0 bytes
```

In [189…
```
df_db_support['complaint_date'] = pd.to_datetime(df_db_support['complaint_date'])
df_db_support.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 4 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   customerid      9 non-null      str
 1   complaint_date  9 non-null      datetime64[us]
 2   escalations     9 non-null      str
 3   csat_score      9 non-null      int64
dtypes: datetime64[us](1), int64(1), str(2)
memory usage: 420.0 bytes
```

In [190…
```
#3. Feature Engineering and data analysis
```

In [191…
```
#create A New column using existing column - churn flag
df_db_subscription['churn flag'] = np.where(df_db_subscription['cancellation_date']
df_db_subscription.head()
```

Out[191…

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	contract
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	A
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	A
2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	M
3	0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	A
4	0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	M

In [192…
```
#first fix support table duplicates then merge
df = (df_db_subscription.merge(df_db_customer, on = 'customerid', how= 'left')
                        .merge(df_db_support, on = 'customerid', how = 'left'))
```

In [193…
```
df.shape
```

Out[193… (23, 20)

In [194…
```
df_db_subscription['customerid'].nunique()
```

Out[194… 21

In [195…
```
df_db_customer['customerid'].nunique()
```

Out[195… 21

In [196…
```
df_db_support['customerid'].nunique()
```

Out[196… 7

In [197…
```
df_db_support['customerid'].size
```

Out[197…
```
9
```

In [198…
```
df_db_support['complaint_count'] = df_db_support.groupby('customerid')['customerid'
```

In [199…
```
df_db_support = df_db_support.sort_values('complaint_date').drop_duplicates('custom
```

In [200…
```
df_db_support['customerid'].size
```

Out[200…
```
7
```

In [201…
```
#merge df
df = (df_db_subscription.merge(df_db_customer, on='customerid', how='left')
                        .merge(df_db_support, on = 'customerid', how='left'))
```

In [202…
```
df.shape
```

Out[202…
```
(21, 21)
```

In [203…
```
df.to_csv('C:/Users/ADMIN/Downloads/customer_churn_data.csv',index=False)
```

In [204…
```
#data analysis
```

In [205…
```
#1. Churn Rate
```

In [206…
```
df.columns
```

Out[206…
```
Index(['customerid', 'subscription_start_date', 'subscription_type',
       'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
       'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
       'churn flag', 'customer_name', 'country', 'state', 'gender', 'dob',
       'complaint_date', 'escalations', 'csat_score', 'complaint_count'],
      dtype='str')
```

In [207…
```
churn_rate = df['churn flag'].mean()*100
print("churn rate = ", round(churn_rate,2), '%')
```

```
churn rate =  28.57 %
```

In [208…
```
print(df.columns.tolist())
```

```
['customerid', 'subscription_start_date', 'subscription_type', 'renewal_date', 'plan
_type', 'contract_type', 'cancellation_date', 'cancellation_reason', 'monthly_charge
s', 'cltv', 'churn_score', 'churn flag', 'customer_name', 'country', 'state', 'gende
r', 'dob', 'complaint_date', 'escalations', 'csat_score', 'complaint_count']
```

In [209…
```
#2. Retenion Rate
retention_rate = 100-churn_rate
print("Retention Rate =", round(retention_rate, 2), "%")
```

```
Retention Rate = 71.43 %
```

In [210…
```
df.head(2)
```

Out[210…

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	contract
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	A
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	A

2 rows × 21 columns

In [211…

```
#3. Churn by plan type
churn_by_plan = df.groupby('plan_type')['churn flag'].mean().mul(100).round(2).rese
print(churn_by_plan)
```

```
  plan_type  churn_rate_pct
0     Basic           60.00
1   Premium           14.29
2  Standard           22.22
```

In [216…

```
#4. a. Churn by state + sum(revenue) and count of users
churn_by_state = df.groupby('state').agg(
    total_revenue=('monthly_charges', 'sum'),
    user_count=('customerid', 'nunique'),
    churn_count=('churn flag', 'sum')
).reset_index()

churn_by_state
```

Out[216…

	state	total_revenue	user_count	churn_count
0	Delhi	52.96	4	1
1	Karnataka	20.98	2	2
2	Kathmandu	20.98	2	0
3	Maharashtra	50.97	3	0
4	Meghalaya	42.97	3	2
5	Nagaland	22.99	1	0
6	Rajasthan	36.98	2	0
7	Telangana	30.98	2	1
8	Uttar Pradesh	115.98	2	0

In [217…

```
#4 b. churn by subscription type + sum(revenue)  and count of users
churn_by_subscription = df.groupby('subscription_type').agg(
    total_revenue=('monthly_charges', 'sum'),
    user_count=('customerid', 'nunique'),
    churn_count=('churn flag', 'sum')
).reset_index()
```

```
churn_by_subscription
```

Out[217…

	subscription_type	total_revenue	user_count	churn_count
0	Organic	145.91	9	0
1	Paid	174.94	6	1
2	Refferal	74.94	6	5

In [219…

```
#5. Avg revenue per user
arpu = df['monthly_charges'].mean()
print('ARPU= ', round(arpu,2))
```

```
ARPU=  18.85
```

In [220…

```
 #6. Avg customer Tenure
#count of days users has used our service : cancellation date else current date
pd.Timestamp.today()
```

Out[220…

```
Timestamp('2026-09-02 20:33:26.639877')
```

In [218…

```
#calculate customer age
from datetime import datetime

df['dob'] = pd.to_datetime(df['dob'], errors='coerce')

df['customer_age'] = (
    datetime.now().year - df['dob'].dt.year
)

df[['dob', 'customer_age']].head()
```

Out[218…

	dob	customer_age
0	1982-04-12	44
1	1995-11-23	31
2	1978-02-15	48
3	2001-08-30	25
4	1990-05-05	36

In [221…

```
today = pd.Timestamp.today()

df['tenure_days'] = np.where(
    df['cancellation_date'].notna(),
    (df['cancellation_date'] - df['subscription_start_date']).dt.days,
    (today - df['subscription_start_date']).dt.days
)
```

```
avg_tenure = df['tenure_days'].mean()
print('Avg tenure (days)=', round(avg_tenure),0)
```

```
Avg tenure (days)= 1530 0
```

In [222…
```
#Revenue of risk - revenue lost from churned users
revenue_at_risk = df.loc[df['churn flag']==1, 'monthly_charges'].sum()
print("revenue_at_risk = ", revenue_at_risk)
```

```
revenue_at_risk =  73.94
```

In [223…
```
#8. Escalation rate
escalation_rate = (df['escalations']=='Y').mean()*100
print("Escalation Rate = ", round(escalation_rate, 2), "%")
```

```
Escalation Rate =  19.05 %
```

In [224…
```
#9. Avg Complaint per User
avg_complaints = df['complaint_count'].sum()/df['customerid'].nunique()
print("Avg Compliants Per User =", round(avg_complaints, 2))
```

```
Avg Compliants Per User = 0.43
```

In [236…
```
#10. Correlation Escalation vs Churn
corr_df = df[['escalations','churn flag']].dropna()
correlation = corr_df['escalations'].corr(df['churn flag'])
print("Correlation between escalation vs churn is =", round(correlation,2))
# df['escalations'] = np.where(df['escalations']=='Y', 1, 0)
```

```
Correlation between escalation vs churn is = nan
```

In [237…
```
#11. churn risk - create a column using existing col
conditions = [
    (df['churn_score'] < 50),
    (df['churn_score'] >= 50) & (df['churn_score'] < 70),
    (df['churn_score'] >= 70)
]
choices = ['low','med','high']
df['churn_risk'] = np.select(conditions, choices, default='unknown')
```

In [238…
```
df[['churn_risk','churn_score']].head()
```

Out[238…

	churn_risk	churn_score
0	low	12
1	high	91
2	low	34
3	low	8
4	high	88

In [239…
```
#4. Visualization using Matplotlib
```

In [240…
```
df_visual = df.copy()
df.head()
```

Out[240…

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	contract
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	A
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	A
2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	M
3	0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	A
4	0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	M

5 rows × 24 columns

In [241…
```
df_visual.columns
```

Out[241…
```
Index(['customerid', 'subscription_start_date', 'subscription_type',
       'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
       'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
       'churn flag', 'customer_name', 'country', 'state', 'gender', 'dob',
       'complaint_date', 'escalations', 'csat_score', 'complaint_count',
       'customer_age', 'tenure_days', 'churn_risk'],
      dtype='str')
```

In [244…
```
print(df_visual['churn flag'].value_counts(dropna=False))
```

```
churn flag
NaN    21
Name: count, dtype: int64
```

In [247…
```
df_visual['churn flag'] == 1
```

Out[247…
```
0     False
1     False
2     False
3     False
4     False
5     False
6     False
7     False
8     False
9     False
10    False
11    False
12    False
13    False
14    False
15    False
16    False
17    False
18    False
19    False
20    False
Name: churn flag, dtype: bool
```

In [250…
```
print(df['churn flag'].head(20))
print(df['churn flag'].dtype)
```

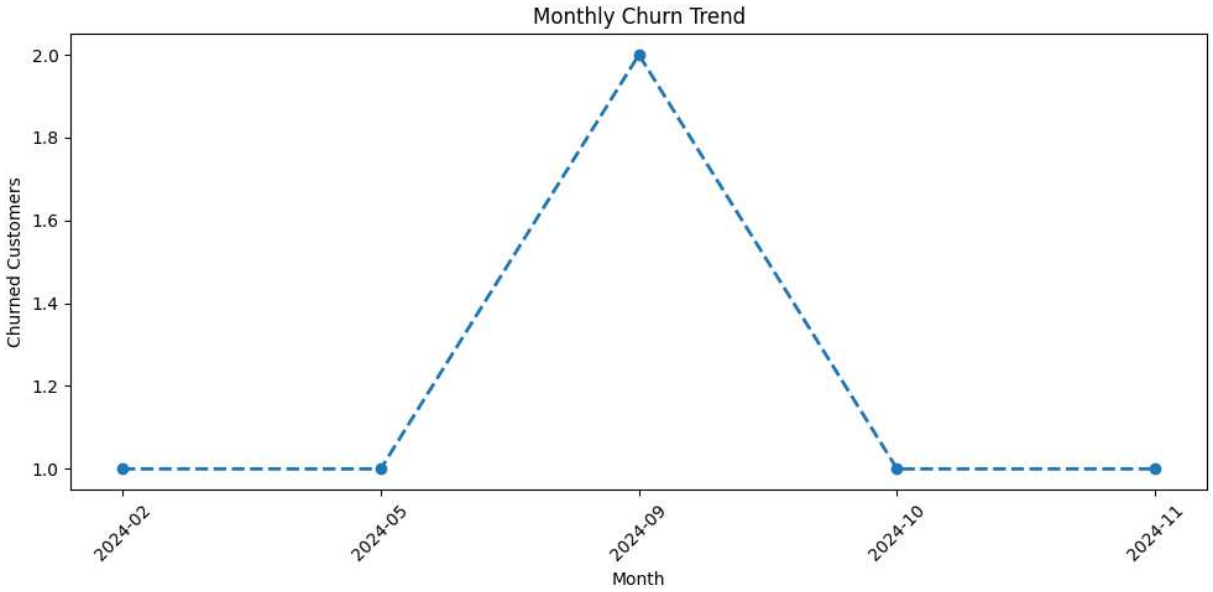
```
0    NaN
1    NaN
2    NaN
3    NaN
4    NaN
5    NaN
6    NaN
7    NaN
8    NaN
9    NaN
10   NaN
11   NaN
12   NaN
13   NaN
14   NaN
15   NaN
16   NaN
17   NaN
18   NaN
19   NaN
Name: churn flag, dtype: float64
float64
```

In [251…
```
df_visual['cancellation_date'] = pd.to_datetime(
    df_visual['cancellation_date'],
    errors='coerce'
)

df_visual['churn flag'] = df_visual['cancellation_date'].notna().astype(int)
```

In [252…
```
print(df_visual['churn flag'].value_counts())
```

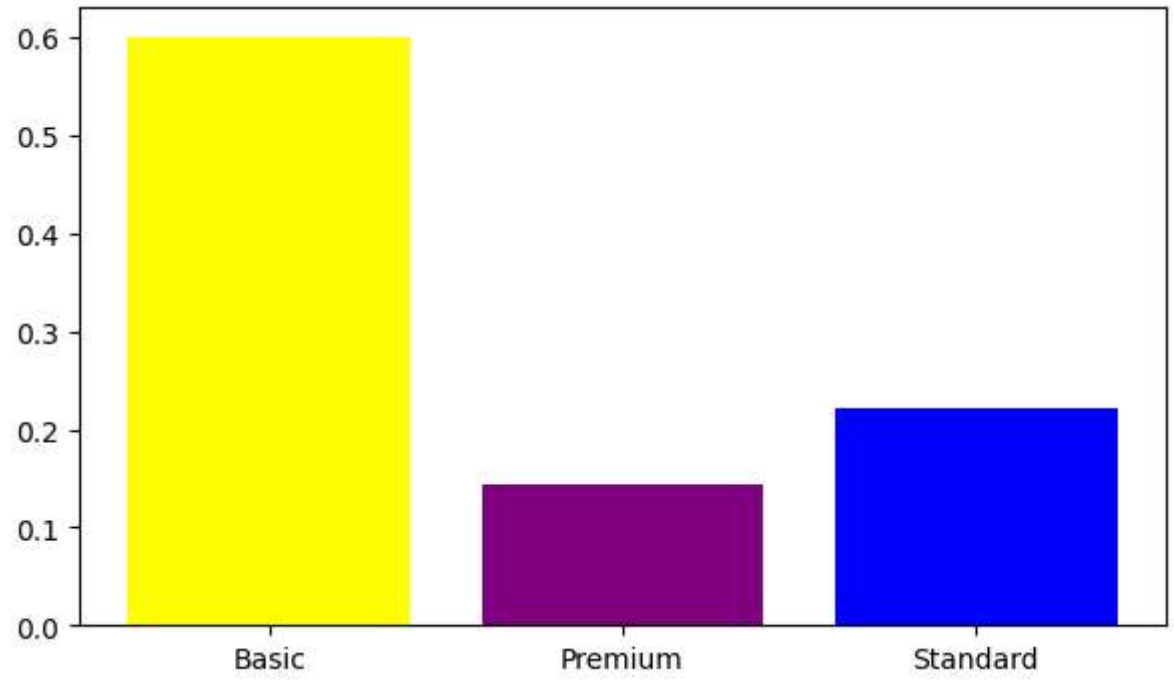
```
churn flag
0    15
1     6
Name: count, dtype: int64
```

In [253…
```
#4.1 Monthly hurn Trend (Time series KPI)
# df_visual['cancellation_month'] = df_visual['cancellation_date'].dt.to_period('M'
# churn_trend = df_visual[df_visual['churn flag'] == 1].groupby('cancellation_month
# plt.figure(figsize=(8,3))
# plt.plot(churn_trend.index.astype('str'), churn_trend.values, color='green' ,mark
# plt.title('Monthly Churn Trend')
# plt.xlabel('Month')
# plt.ylabel('churned Customers')
# plt.show()

df_visual['cancellation_month'] = df_visual['cancellation_date'].dt.to_period('M')

churn_trend = (
    df_visual[df_visual['churn flag'] == 1]
    .groupby('cancellation_month')
    .size()
)

plt.figure(figsize=(10, 5))
plt.plot(
    churn_trend.index.astype(str),
    churn_trend.values,
    marker='o',
    linestyle='dashed',
    linewidth=2
)

plt.title('Monthly Churn Trend')
plt.xlabel('Month')
plt.ylabel('Churned Customers')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



In [ ]:

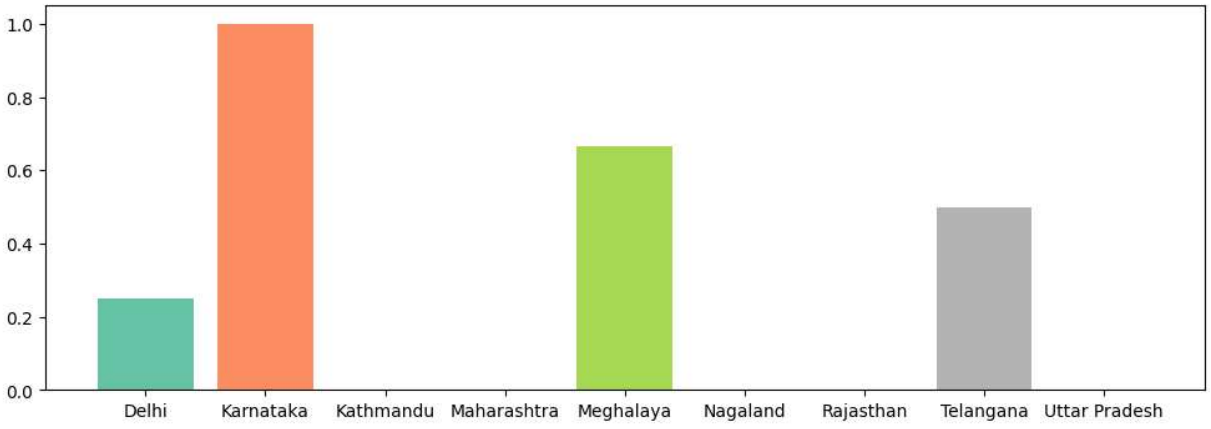
In [254…
```
#4.2 churn by plan type
churn_plan=df_visual.groupby('plan_type')['churn flag'].mean()
colors = ['yellow','purple','Blue']
plt.figure(figsize=(7,4))
plt.bar(churn_plan.index , churn_plan.values, color=colors)
plt.show()
```



In [255…
```
print(df.columns.tolist())
```

```
['customerid', 'subscription_start_date', 'subscription_type', 'renewal_date', 'plan
_type', 'contract_type', 'cancellation_date', 'cancellation_reason', 'monthly_charge
s', 'cltv', 'churn_score', 'churn flag', 'customer_name', 'country', 'state', 'gende
r', 'dob', 'complaint_date', 'escalations', 'csat_score', 'complaint_count', 'custom
er_age', 'tenure_days', 'churn_risk']
```

In [256…
```
#4.3 churn by states
churn_plan = df_visual.groupby('state')['churn flag'].mean()
colors = plt.cm.Set2(np.linspace(0, 1, len(churn_plan)))
plt.figure(figsize=(12,4))
plt.bar(churn_plan.index, churn_plan.values, color = colors)
plt.show()
```



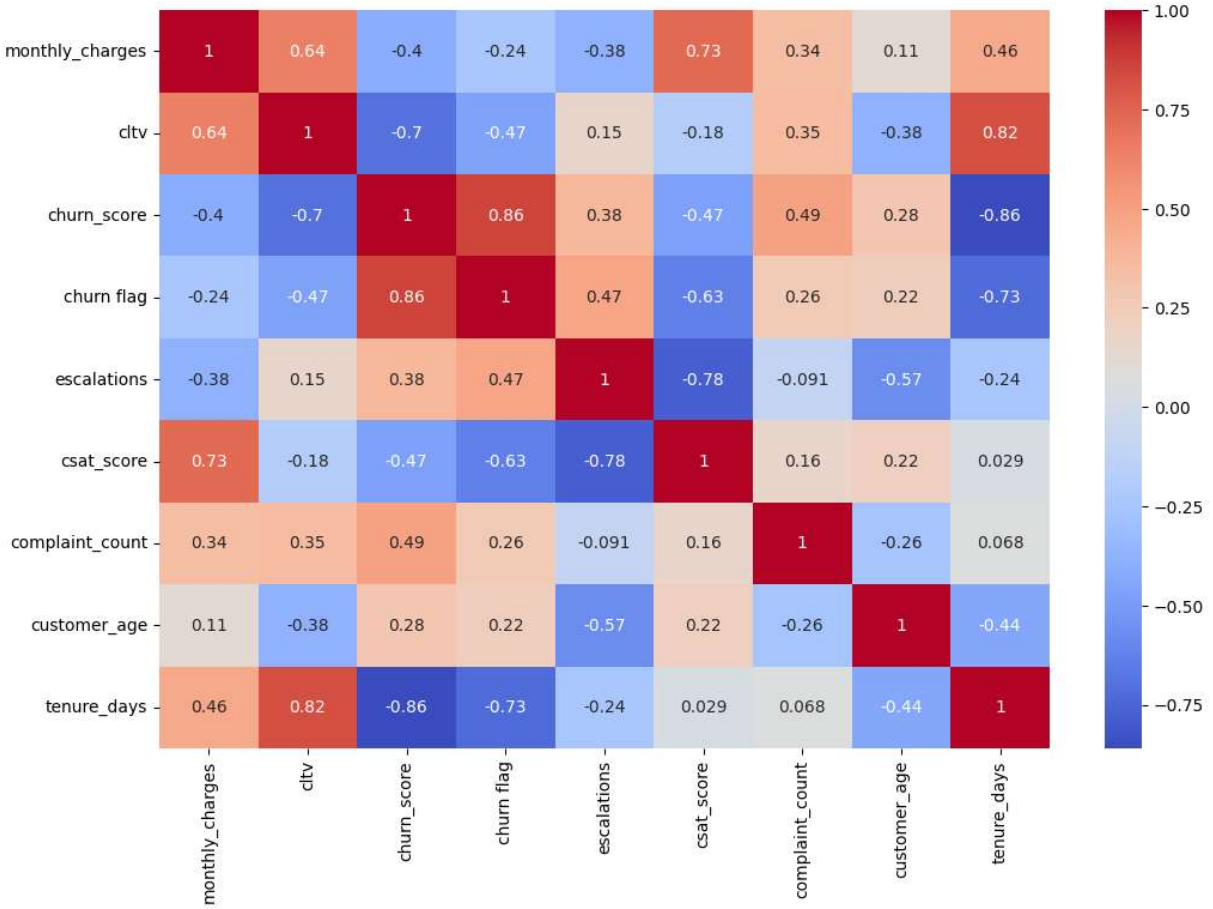
In [257…
```
#Visualization using seaborn
```

In [258…
```
#Encoding - Convert str to numeric so that we can find corr between features
df_visual.columns
```

Out[258…
```
Index(['customerid', 'subscription_start_date', 'subscription_type',
       'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
       'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
       'churn flag', 'customer_name', 'country', 'state', 'gender', 'dob',
       'complaint_date', 'escalations', 'csat_score', 'complaint_count',
       'customer_age', 'tenure_days', 'churn_risk', 'cancellation_month'],
      dtype='str')
```

In [259…
```
df_encoded = df_visual[['plan_type', 'contract_type', 'churn_score', 'churn flag',
categorial_cols =['plan_type', 'contract_type', 'churn_risk']

for col in categorial_cols:
    df_encoded[col] = df_encoded[col].astype('category').cat.codes
```

In [260…
```
#Heatmap (correlation matrix)
import matplotlib.pyplot as plt
import seaborn as sns

numeric_df = df_visual.select_dtypes(include='number')

plt.figure(figsize=(12, 8))
sns.heatmap(numeric_df.corr(), annot=True, cmap='coolwarm')
plt.show()
```



In [261…
```
df_visual[['plan_type', 'contract_type', 'churn_score', 'churn flag','escalations',
```

Out[261…

	plan_type	contract_type	churn_score	churn flag	escalations	churn_risk
0	Standard	Annual	12	0	NaN	low
1	Premium	Annual	91	1	1.0	high
2	Basic	Monthly	34	0	NaN	low
3	Premium	Annual	8	0	NaN	low
4	Standard	Monthly	88	1	1.0	high

In [262…
```
df_encoded.head()
```

Out[262…

	plan_type	contract_type	churn_score	churn flag	escalations	churn_risk
0	2	0	12	0	NaN	1
1	1	0	91	1	1.0	0
2	0	1	34	0	NaN	1
3	1	0	8	0	NaN	1
4	2	1	88	1	1.0	0

In [263…
```
import warnings
warnings.filterwarnings("ignore")
#Correct method of encoding - based on priority

# Define priority order
order_mappings = {
    'plan_type': ['Basic', 'Standard', 'Premium'],
    'contract_type': ['Monthly', 'Annual'],
    'churn_risk': ['Low', 'med', 'high']
}

# Apply priority encoding
for col, order in order_mappings.items():
    df_encoded[col] = pd.Categorical(
        df_encoded[col],
        categories=order,
        ordered=True
    ).codes
```

In [264…
```
df_visual[['plan_type','contract_type','churn_score','churn flag','churn_risk','esc
```

Out[264…

	plan_type	contract_type	churn_score	churn flag	churn_risk	escalations
0	Standard	Annual	12	0	low	NaN
1	Premium	Annual	91	1	high	1.0
2	Basic	Monthly	34	0	low	NaN
3	Premium	Annual	8	0	low	NaN
4	Standard	Monthly	88	1	high	1.0

In [265…
```
df_encoded.head()
```

Out[265…

	plan_type	contract_type	churn_score	churn flag	escalations	churn_risk
0	-1	-1	12	0	NaN	-1
1	-1	-1	91	1	1.0	-1
2	-1	-1	34	0	NaN	-1
3	-1	-1	8	0	NaN	-1
4	-1	-1	88	1	1.0	-1

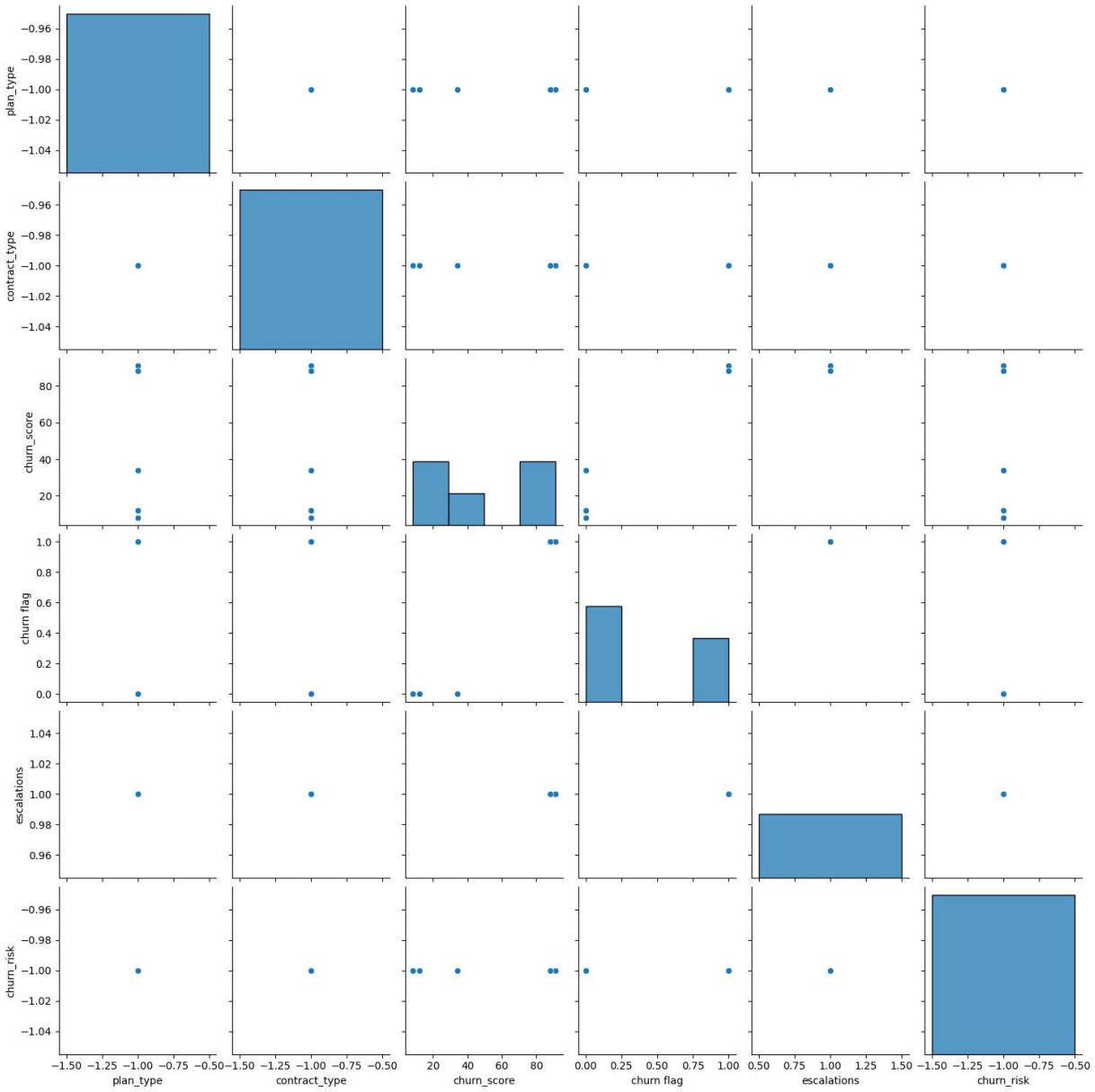
In [266…
```
print(df_visual['plan_type'].unique())
print(df_visual['contract_type'].unique())
print(df_visual['churn_risk'].unique())
print(df_visual['churn flag'].unique())
print(df_visual['escalations'].unique())
```

```
<StringArray>
['Standard', 'Premium', 'Basic']
Length: 3, dtype: str
<StringArray>
['Annual', 'Monthly']
Length: 2, dtype: str
<StringArray>
['low', 'high', 'med']
Length: 3, dtype: str
[0 1]
[nan  1.  0.]
```

In [267…

```
#pairplot
import seaborn as sns
sns.pairplot(df_encoded)
```

Out[267… <seaborn.axisgrid.PairGrid at 0x1990cf25eb0>



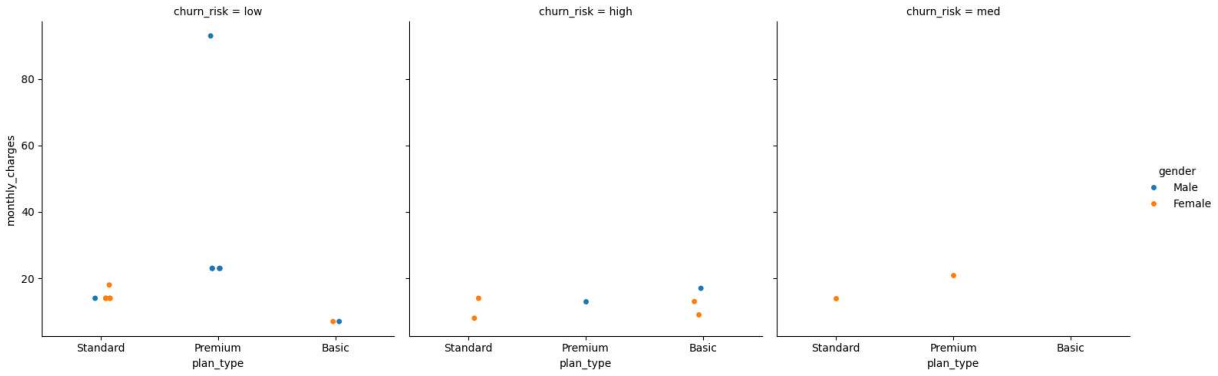
In [270…

```
df_visual.columns
```

Out[270…
```
Index(['customerid', 'subscription_start_date', 'subscription_type',
       'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
       'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
       'churn flag', 'customer_name', 'country', 'state', 'gender', 'dob',
       'complaint_date', 'escalations', 'csat_score', 'complaint_count',
       'customer_age', 'tenure_days', 'churn_risk', 'cancellation_month'],
      dtype='str')
```

In [271…
```
#catplt/facegrid plot - multi-dim comparison
sns.catplot(data=df_visual,
            x='plan_type',
            y='monthly_charges',
            hue='gender',
            col='churn_risk')
```

Out[271…
```
<seaborn.axisgrid.FacetGrid at 0x1990f258d70>
```



In [272…
```
#pivot table
```

In [275…
```
pd.pivot_table(
    df_visual,
    values='churn flag',
    index='plan_type',
    aggfunc = 'mean'
).reset_index()
```

Out[275…

	plan_type	churn flag
0	Basic	0.600000
1	Premium	0.142857
2	Standard	0.222222

In [276…
```
df_visual.columns
```

Out[276…
```
Index(['customerid', 'subscription_start_date', 'subscription_type',
       'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
       'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
       'churn flag', 'customer_name', 'country', 'state', 'gender', 'dob',
       'complaint_date', 'escalations', 'csat_score', 'complaint_count',
       'customer_age', 'tenure_days', 'churn_risk', 'cancellation_month'],
      dtype='str')
```

In [278…
```
pd.pivot_table(
    df_visual,
    index='plan_type',
    values=['monthly_charges','customerid','churn flag'],
    aggfunc={
        'monthly_charges':'sum',
        'customerid':'nunique',
        'churn flag':'mean'
    }

)
```

Out[278…

plan_type	churn flag	customerid	monthly_charges
Basic	0.600000	5	52.95
Premium	0.142857	7	218.93
Standard	0.222222	9	123.91

In [279…
```
#working with SQL in python(pandas)
```

In [280…
```
#create db in sql
# conn = sqlite3.connect('test_database1.sqlite')

# #table details
# conn.execute("create table users(first_name TEXT, country Text, budget integer)")

# #commit and save
# conn.commit()

import sqlite3
import pandas as pd

conn = sqlite3.connect("test_database.sqlite")
cursor = conn.cursor()
```

In [281…
```
#insert data
# cursor = conn.cursor()

# cursor.execute("""
#     INSERT INTO users (first_name, country, budget)
#     VALUES
#     ('Madhav', 'India', 5000),
#     ('Rishabh', 'German', 2500),
#     ('Vishakha', 'India', 3500)
# """)

# conn.commit()
# print("Data Inserted Succesfully")
```

```
cursor.execute("SELECT * FROM users")
print(cursor.fetchall())
```

```
[('Madhav', 'India', 5000), ('Rishabh', 'German', 2500), ('Vishakha', 'India', 350
0)]
```

In [282…

```
#Check inserted data in table
# query = "SELECT * FROM users"

# df_results = pd.read_sql(query, conn)

# df_results

cursor.execute("""
    INSERT INTO users (first_name, country, budget)
    VALUES
    ('Madhav', 'India', 5000),
    ('Rishabh', 'German', 2500),
    ('Vishakha', 'India', 3500)
""")

conn.commit()
```

In [283…

```
cursor.execute("SELECT * FROM users")
print(cursor.fetchall())
```

```
[('Madhav', 'India', 5000), ('Rishabh', 'German', 2500), ('Vishakha', 'India', 350
0), ('Madhav', 'India', 5000), ('Rishabh', 'German', 2500), ('Vishakha', 'India', 35
00)]
```

In [284…

```
query = "SELECT * FROM users"

df_results = pd.read_sql(query, conn)

df_results
```

Out[284…

	first_name	country	budget
0	Madhav	India	5000
1	Rishabh	German	2500
2	Vishakha	India	3500
3	Madhav	India	5000
4	Rishabh	German	2500
5	Vishakha	India	3500

In [285…

```
#aggreagation
query = """
          select country, sum(budget) as total_budget
          from users
          Group by country
"""
```

```
df_agg = pd.read_sql(query, conn)
df_agg
```

Out[285…

	country	total_budget
0	German	5000
1	India	17000

In [286…

```
conn.close()
```

In [ ]: